Cardiovascular Disease Prediction

Li Lynn Sheryl Phoa, Yilin Fang, Yuet Wah Chan

***Abstract*—This project addresses the prediction of Cardiovascular Diseases using the Behavioral Risk Factor Surveillance System (BRFSS) dataset. The dataset contains over 300,000 survey responses with 300+ categorical features. A major challenge is the severe class imbalance (only 8.5% positive class), which makes raw accuracy a misleading metric. We implement and compare logistic regression models from scratch. By employing undersampling and L_2 regularization ($\lambda = 1.0$), we successfully increase the F1-score from 0.128 to 0.354 on cross-validation, achieving 0.360 on the AIcrowd test set.**

I. Introduction

Cardiovascular diseases are a leading cause of death globally. Predicting heart disease from lifestyle and clinical survey data can aid in early detection. In this project, we use data from the Behavioral Risk Factor Surveillance System (BRFSS) to classify whether a respondent has coronary heart disease.

The primary difficulty of this dataset is class imbalance: 91.5% of respondents are healthy. A naive model predicting "No Disease" achieves 91.5% accuracy but is clinically useless. Therefore, we optimize for the F1-score to balance precision and recall.

II. Methods and Models

A. Data Preparation & Feature Generation

To prepare the data, we applied the following pipeline:

- **Metadata Removal:** We dropped the first 13 columns (e.g., `SEQNO`, `IDATE`) as they are survey identifiers and hold no medical predictive power.
- **Missing Values:** Empty CSV fields parsed as `NaN`. We replaced all `NaN` and `Inf` values with 0.0.
- **Standardization:** Features were standardized to zero mean and unit variance to prevent gradient explosion.
- **Bias:** A constant column of 1s was appended to the feature matrix.

B. Undersampling

To combat class imbalance, we randomly undersampled the majority class in the training folds only, creating a 1:1 balanced training set. This prevents the model from simply predicting the majority class.

C. Regularized Logistic Regression

We minimized the negative log-likelihood with an L_2 penalty:

$$L(w) = \frac{1}{N}\sum_{n=1}^{N}\left[-y_n x_n^T w + \log(1 + e^{x_n^T w})\right] + \lambda\|w\|^2 \quad (1)$$

We trained the model using Gradient Descent with step size $\gamma = 0.1$ for 1000 iterations. The gradient update is:

$$w_{t+1} = w_t - \gamma\left(\frac{1}{N}X^T(\sigma(Xw_t) - y) + 2\lambda w_t\right) \quad (2)$$

D. Cross-Validation

We used 5-fold cross-validation. To prevent data leakage, undersampling and standardization were fit *only* on the 4 training folds, and then applied to the validation fold.

III. Results and Discussion

A. Ablation Study

We first evaluated the impact of undersampling using standard logistic regression. As seen in Table I, undersampling drops the overall accuracy but massively improves the F1-score and Recall by actually detecting sick patients.

Model	Accuracy	Recall	F1
No Undersampling	91.36%	0.18	0.128
With Undersampling	74.97%	0.65	0.307

TABLE I
Impact of undersampling on 5-fold CV.

Because our model was overfitting on the 308 features, we added L_2 regularization. Table II shows our hyperparameter sweep. We found $\lambda = 1.0$ to be optimal, yielding the best balance of bias and variance.

Hyperparameter λ	Accuracy	F1-Score
0.01	77.87%	0.322
0.1	80.81%	0.350
1.0	**87.06%**	**0.354**
10.0	51.08%	0.105

TABLE II
Ablation study of L_2 regularization strength.

B. Final Performance

Our best model (Regularized Logistic Regression, $\lambda = 1.0$, undersampled) achieved an accuracy of 86.7% and an F1-score of 0.360 on the AIcrowd test set. The significant improvement over the baseline (0.128) demonstrates that proper data preprocessing and regularization are critical for imbalanced medical datasets.

IV. Summary

We successfully applied undersampling and L_2 regularization to train a logistic regression model that effectively identifies heart disease in the BRFSS dataset. By prioritizing the F1-score over accuracy, our model is clinically more meaningful. All code, cross-validation steps, and the `run.py` pipeline are available in our public GitHub repository to ensure reproducibility.